

NISAR

Submitted By,

Sreejata Bhattacharjee

Roll no: 25CEM5R10



Submitted To,

Dr Narayanarao

Thermal, Microwave and hyperspectral Remote Sensing

Understanding the Backscattered image of NISAR Data using GCOV and GSLC product

NISAR (NASA–ISRO Synthetic Aperture Radar) is an Earth-observation satellite mission jointly developed by **NASA** and the **Indian Space Research Organisation (ISRO)**. It is one of the most advanced radar-imaging satellites designed to study changes on Earth's surface with very high precision.

1. What is NISAR?

NISAR is a satellite that uses **Synthetic Aperture Radar (SAR)** to map the Earth's surface. Unlike optical satellites, radar can:

- See **through clouds**
- Work **day and night**
- Detect **very small ground movements** (even a few millimeters)

A unique feature of NISAR is that it carries **two radar systems**:

- **L-band radar** (provided by NASA) – good for studying forests, soil moisture, and large-scale land deformation
- **S-band radar** (provided by ISRO) – useful for agriculture, soil conditions, and surface changes

This dual-frequency system allows scientists to get more accurate and complementary data than a single radar band.

2. Why was NISAR built?

NISAR was built mainly because scientists need **continuous, precise, and global measurements** of how Earth's surface is changing. Many natural and human-induced processes happen slowly and are difficult to detect without long-term monitoring.

Major reasons include:

1. **Increasing natural disasters**
 - Earthquakes
 - Landslides
 - Floods
 - Volcanoes
2. **Climate change impacts**
 - Melting glaciers and ice sheets
 - Sea-level rise
 - Coastal erosion
3. **Environmental monitoring**
 - Deforestation
 - Wetland changes
 - Soil moisture and drought
4. **Infrastructure and urban growth**

- Ground subsidence in cities
- Dam and reservoir monitoring

3. Purpose of NISAR (in detail)

A. Monitoring Earth's Surface Deformation

NISAR can detect **tiny ground movements** caused by:

- Tectonic plate motion
- Earthquakes and fault activity
- Volcano inflation or deflation

This helps scientists:

- Understand earthquake hazards
- Improve early warning systems

B. Studying Forests and Biomass

The satellite can measure:

- Forest density
- Tree height and biomass
- Deforestation rates

This helps in:

- Carbon cycle studies
- Climate change research

C. Ice and Glacier Monitoring

NISAR can observe:

- Ice sheet movement in polar regions
- Glacier retreat
- Ice thickness changes

This helps predict:

- Sea-level rise
- Climate trends

D. Agriculture and Soil Moisture

Radar can estimate:

- Soil moisture levels
- Crop conditions

- Flooded agricultural land

This supports:

- Food security planning
- Drought assessment

E. Disaster Management

NISAR data will help in:

- Flood mapping
- Landslide detection
- Damage assessment after earthquakes

Because radar works through clouds, it is especially useful during storms and monsoon conditions.

F. Coastal and Ocean Studies

It can track:

- Shoreline changes
- Coastal erosion
- Wetlands and mangroves

This is important for countries like India with long coastlines.

4. Why the NASA–ISRO Collaboration Matters

NISAR is significant because:

- It combines **advanced US radar technology** with **Indian satellite and launch capabilities**.
- It represents one of the **largest Earth-science collaborations** between two space agencies.
- Data will be **openly available to scientists worldwide**, increasing global research.

5. How NISAR Works (Simple Explanation)

1. The satellite sends **microwave radar signals** toward Earth.
2. The signals bounce back from the ground.
3. By comparing repeated images, scientists can detect:
 - Movement
 - Changes in vegetation
 - Surface moisture

This method is called **Interferometric SAR (InSAR)**.

6. Importance for India and the World

For India specifically, NISAR is valuable for:

- Monitoring **Himalayan landslides and glaciers**
- Tracking **floods during monsoon**
- Observing **agriculture and drought**
- Studying **coastal erosion**

Globally, it helps in:

- Climate research
- Disaster preparedness
- Environmental protection

A **backscatter image** in **NISAR data** is a radar image that shows **how much of the radar signal sent by the satellite is reflected (scattered back) to the sensor from the Earth's surface**. It is one of the primary data products generated by the **NASA-ISRO Synthetic Aperture Radar mission**, developed by **NASA** and **Indian Space Research Organisation (ISRO)**.

1. Definition of Backscatter

Backscatter is the portion of electromagnetic energy transmitted by a radar system that is **reflected back toward the antenna** after interacting with the surface.

In radar remote sensing:

- The satellite sends **microwave pulses** to Earth.
- Different surfaces reflect different amounts of energy.
- The returned signal strength is recorded and converted into an image.

A **backscatter image** therefore represents **the intensity of returned radar energy for each pixel**.

2. What a Backscatter Image Shows

In a NISAR backscatter image:

- **Bright areas (high backscatter)**
Strong reflection
Examples:
 - Buildings and urban areas
 - Rough terrain
 - Dense forests
- **Dark areas (low backscatter)**
Weak reflection
Examples:
 - Calm water bodies
 - Smooth surfaces
 - Dry, flat soil

So the brightness in the image does **not represent color or temperature**, but **radar reflectivity**.

3. Why Backscatter Occurs (Physical Reasons)

Backscatter depends mainly on three factors:

A. Surface Roughness

- Rough surfaces scatter more energy → brighter image
- Smooth surfaces reflect energy away → darker image

B. Moisture Content

Water increases dielectric constant:

- Wet soil → high backscatter
- Dry soil → low backscatter

This is why radar is useful for **soil moisture estimation**.

C. Geometry and Structure

- Forest canopy causes **volume scattering**
- Buildings cause **double-bounce scattering**
- Flat ground causes **surface scattering**

4. Measurement of Backscatter

Backscatter is usually expressed as:

Sigma naught (σ^0)

It represents:

- Radar cross-section per unit area
- Measured in **decibels (dB)**

Typical values:

- Water: -25 to -15 dB
- Bare soil: -15 to -5 dB
- Urban areas: -5 to +5 dB

5. Polarization in NISAR Backscatter Images

NISAR records backscatter in **different polarizations**, such as:

- HH
- HV

- VV
- VH

Each polarization provides different information about:

- Vegetation structure
- Soil condition
- Surface geometry

For example:

- Cross-polarization (HV or VH) is sensitive to **vegetation**.
- Co-polarization (HH or VV) is useful for **surface and soil analysis**.

5. Types of Backscatter Images in NISAR Data

Common backscatter products include:

1. **SingleLookComplex(SLC)**
Contains amplitude and phase information
2. Used for interferometry
3. **Detected Backscatter Image (Amplitude image)**
Used for interpretation and mapping
4. **Radiometrically calibrated backscatter (σ^0 images)**
Used for scientific analysis

7. Applications of Backscatter Images

Backscatter images from NISAR are used for:

Agriculture

- Crop growth monitoring
- Soil moisture estimation

Forestry

- Biomass estimation
- Deforestation detection

Disaster Management

- Flood mapping
- Landslide detection

Urban Studies

- Land-use classification
- Infrastructure monitoring

Cryosphere Studies

- Glacier movement
- Snow and ice mapping

8. Simple Interpretation Example

If you see a radar backscatter image of a region:

- A river appears **dark** because smooth water reflects radar away.
- A city appears **bright** due to buildings reflecting signals strongly.
- Forests appear **moderately bright** due to canopy scattering.

9. Why Backscatter Images Are Important

Backscatter images are valuable because they:

- Work **day and night**
- Work **through clouds and monsoon weather**
- Detect **surface changes over time**

This makes them ideal for long-term environmental monitoring.

Below is the **complete step-by-step procedure to work with a GCOV L2 SAR (.h5) file in QGIS**, from importing the data to visualization and basic analysis. This workflow applies to **NISAR or similar SAR Level-2 products** provided in **HDF5 (.h5)** format, produced by **NASA and Indian Space Research Organisation (ISRO)**.

Procedure to Process GCOV L2 SAR Image in QGIS (Starting from .h5 File)

Step 1: Install Required Software

Ensure you have:

- QGIS (latest stable version recommended)
- GDAL (installed automatically with QGIS)

Optional but useful:

- SNAP software (for advanced SAR processing)

Step 2: Open QGIS

1. Launch **QGIS Desktop**
2. Create a **new project**
3. Set project CRS if needed (commonly WGS84 EPSG:4326)

Step 3: Import the .h5 File

SAR GCOV Level-2 files are in **HDF5 format**, which contains multiple datasets.

Method 1: Using Data Source Manager

1. Click **Layer** → **Add Layer** → **Add Raster Layer**
2. Browse to your **.h5 file**
3. Select the file
4. Click **Open**

QGIS will list multiple subdatasets.

Step 4: Select Required Dataset

You will see layers such as:

- Backscatter HH
- Backscatter HV
- Backscatter VV
- Incidence angle
- Mask layers

Steps:

1. Choose required polarization (for example HH or VV)
2. Click **Add**

The raster will appear in the Layers panel.

Step 5: Check Projection and Metadata

1. Right-click layer
2. Select **Properties**
3. Open:
 - Information tab
 - Metadata tab

Check:

- CRS
- Pixel size
- Backscatter units (σ^0 in dB)

Step 6: Apply Contrast Stretch for Better Visualization

SAR images initially appear dark.

Steps:

1. Right-click layer → **Properties**
2. Go to **Symbology**
3. Select:
 - Render type: Singleband gray
4. Choose:
 - Min/Max stretch
 - Cumulative count cut (2%–98%)

Click **Apply**

Image becomes clearer.

Step 7: Convert Backscatter to dB (If Required)

Sometimes values are in linear scale.

Steps:

1. Open **Raster Calculator**
2. Use formula:

$10 * \log_{10}(\text{"layer_name"})$

3. Save output as new raster

This converts values to decibels.

Step 8: Clip to Study Area Boundary (Optional)

If studying a specific region:

1. Load boundary shapefile
2. Go to: **Raster → Extraction → Clip Raster by Mask Layer**
3. Select:
 - Input raster
 - Mask layer
4. Run

Output will be clipped image.

Step 9: Create False Color Composite (Polarization Combination)

If multiple polarizations exist:

1. Open: **Raster → Miscellaneous → Build Virtual Raster**
2. Select HH, HV, VV

3. Set:
 - Separate bands
4. Load result

Then:

1. Right-click layer → Symbology
2. Select **Multiband color**
3. Assign:
 - Red: HH
 - Green: HV
 - Blue: VV

This helps visualize scattering mechanisms.

Step 10: Noise Reduction (Optional)

To reduce speckle:

1. Open:
Processing Toolbox
2. Search:
 - Focal statistics (mean filter)

Or process in SNAP for advanced filtering.

Step 11: Extract Pixel Values and Statistics

1. Open:
Raster → Zonal Statistics
2. Select:
 - Raster layer
 - Polygon boundary

Results give:

- Mean backscatter
- Standard deviation

Useful for analysis.

Step 12: Create Histogram of Backscatter

1. Right-click layer → Properties
2. Select **Histogram**
3. Click **Compute Histogram**

This helps interpret land cover.

Step 13: Interpretation of Results

Typical observations:

- Water appears dark
- Vegetation moderate brightness
- Urban bright
- Wet soil brighter than dry soil

Step 14: Export Final Image

1. Right-click raster
2. Select **Export** → **Save As**
3. Choose:
 - GeoTIFF
 - CRS
 - Resolution

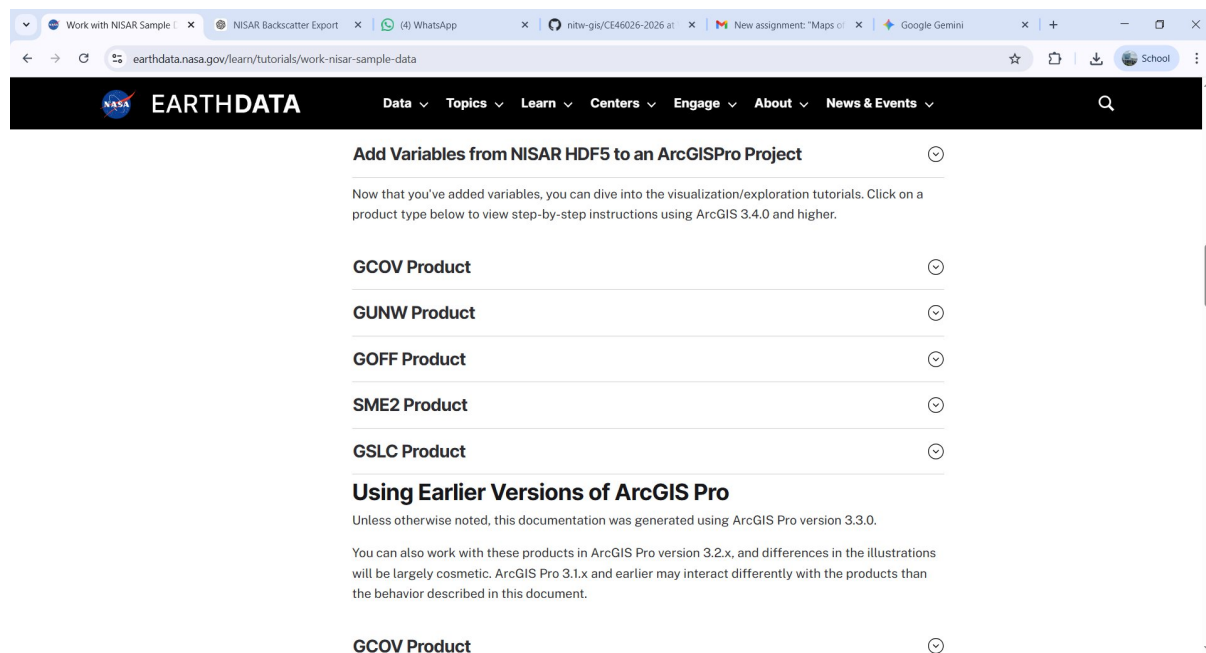
Save output.

Flow Summary (Quick Overview)

1. Open QGIS
2. Import .h5 file
3. Select subdataset
4. Apply contrast stretch
5. Convert to dB (if needed)
6. Clip to boundary
7. Visualize polarizations
8. Analyze and interpret

Practical Observations While Working with GCOV in QGIS

- Image may appear noisy due to speckle
- Polarization bands give different land responses
- Brightness varies with moisture and roughness
- Histogram helps determine threshold for classification



NISAR Data products

There are two sample Geocoded Single-Look Complex (GSLC) products available, one for each of the acquisitions used in other sample products that require imagery pairs:

- NISAR_L2_PR_GSLC_001_030_A_019_2000_SHNA_A_20081012T060910_20081012T060926_D00402_F_N_J_001.h4
- NISAR_L2_PR_GSLC_002_030_A_019_2000_SHNA_A_20081127T060959_20081127T061015_D00402_F_N_J_001.h3

The illustrations in this section use the acquisition from October 12, 2008. Note that the data used to create these sample products are from JAXA's PALSAR sensor, which also operates in the L band, but the product has a smaller footprint than the NISAR acquisitions.

Add GSLC Data

As with the wrapped phase variable in the GUNW products, the SLC variables in the GSLC are in complex data format. While you can add these variables using the Drag and Drop or Add Multidimensional Raster approaches, these methods will display the Amplitude measurements by default, and you will not have access to the phase measurements.

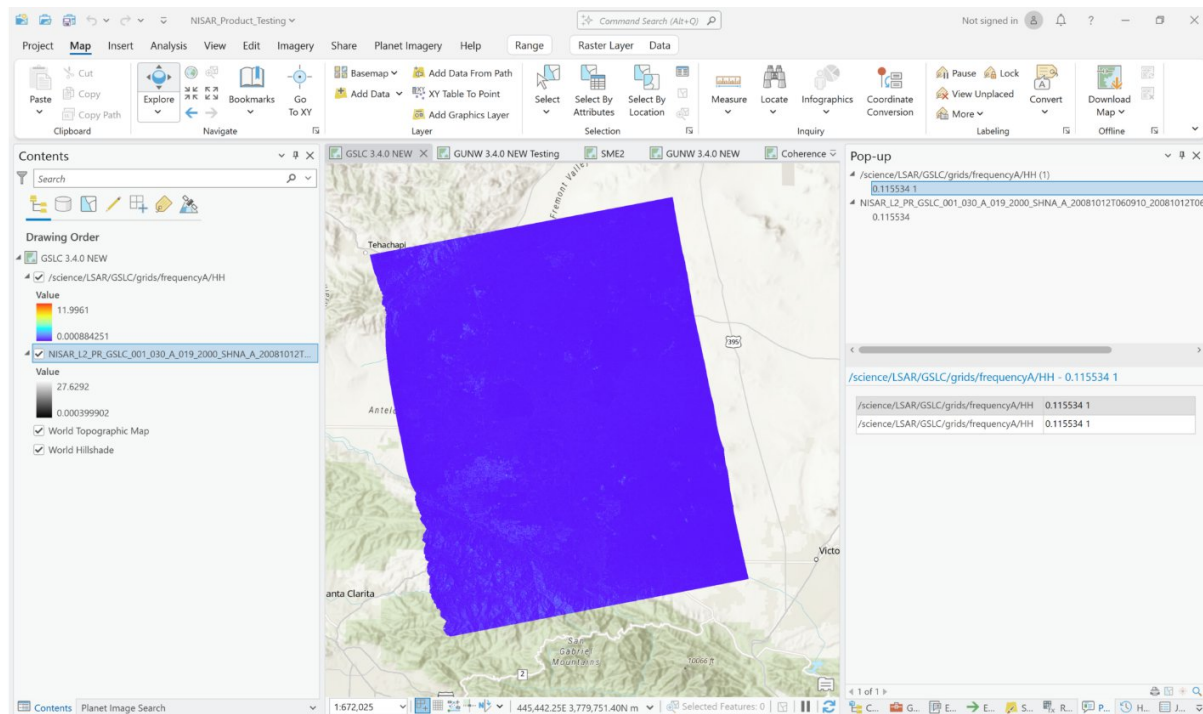
To extract the phase values, you will need to add the SLC variables as Multiband Rasters, so we'll start with that method, then go over the rest of the options.

Add Multiband Rasters

In the Map tab, click the down arrow next to the Add Data button and select Add Multidimensional Data. Navigate to the GSLC HDF5 file, and select it to open the list of variables. Select the focused SLC variable. Click the drop-down menu under Output Configuration and select Multiband Raster, then click OK to add the variable.

The screenshot displays the QGIS desktop environment. The top toolbar shows the 'Add Data' button (a yellow folder icon) highlighted with a red rectangle. Below the toolbar, the 'Contents' panel on the left shows a list of layers, with 'NISAR' selected. The main canvas area shows a satellite image of a coastal region. On the right side, the 'Properties' panel is open, showing the 'Multiband Raster' tab. The 'Output Configuration' section is highlighted with a red rectangle, showing the 'Multiband Raster' output type. The 'Add Multidimensional Data' dialog box is open in the center, showing a table of variables and their descriptions. The 'Output Configuration' section is also visible, showing the 'Multiband Raster' output type.

14



Adding GSLC image parameters

Characteristics of Backscatter Image of NISAR:

Backscatter images from the **NASA-ISRO Synthetic Aperture Radar mission**, developed by **NASA** and **Indian Space Research Organisation (ISRO)**, show how radar energy is reflected from the Earth's surface. These images have several distinctive characteristics that make them different from optical satellite images.

1. Brightness Represents Radar Reflectivity, Not Color

In a backscatter image:

- Bright pixels → strong radar return
- Dark pixels → weak radar return

Brightness depends on:

- Surface roughness
- Moisture content
- Structure and geometry

Example interpretation

- Water bodies → dark
- Agricultural land → gray
- Cities → bright

Diagram:

Download conceptual diagram – Backscatter intensity

2. Dependence on Surface Roughness

One of the most important characteristics of NISAR backscatter images is sensitivity to **surface roughness**.

- Smooth surfaces reflect radar away from the sensor → low backscatter
- Rough surfaces scatter radar in many directions → high backscatter

This makes radar useful for:

- Soil condition studies
- Land surface classification

3. Sensitivity to Soil Moisture

Radar waves interact strongly with water.

Characteristics:

- Wet soil → higher dielectric constant → brighter image
- Dry soil → lower reflection → darker image

This property is important for:

- Agriculture monitoring
- Drought studies
- Flood mapping

NISAR's L-band radar is especially effective for soil moisture and vegetation analysis.

4. Influence of Surface Geometry and Structure

Backscatter depends on the physical structure of objects.

Three major scattering mechanisms occur:

A. Surface Scattering

Occurs on:

- Bare soil
- Roads
- Calm water

Produces moderate or low backscatter.

B. Volume Scattering

Occurs in:

- Forests
- Crops
- Vegetation

Radar waves penetrate canopy and scatter multiple times.

C. Double-Bounce Scattering

Occurs in:

- Urban areas
- Buildings
- Tree trunks over wet soil

Produces very bright returns.

Diagram:

Download conceptual diagram – Scattering mechanisms

5. Presence of Speckle Noise

Backscatter images have a granular texture called **speckle**.

Characteristics:

- Appears as salt-and-pepper noise
- Caused by interference of scattered waves
- Not measurement error but a physical effect

Speckle is reduced using:

- Multi-looking
- Filters (Lee, Frost, Gamma-MAP)

6. Multi-Polarization Information

NISAR collects backscatter in multiple polarizations:

- HH
- HV
- VV
- VH

Characteristics:

- Cross-polarization → vegetation structure

- Co-polarization → surface roughness and soil

This improves:

- Land-cover classification
- Crop analysis
- Forest biomass estimation

7. Radiometric Calibration

Backscatter values are calibrated and expressed as:

Sigma naught (σ^0) in decibels (dB)

Typical ranges:

- Water: -25 to -15 dB
- Soil: -15 to -5 dB
- Urban areas: -5 to +5 dB

This allows:

- Quantitative comparison
- Time-series monitoring

8. Ability to Detect Surface Changes Over Time

A key characteristic of NISAR backscatter images is **temporal sensitivity**.

By comparing images:

- Land subsidence can be detected
- Flood extent can be mapped
- Crop growth stages can be monitored

This is important for:

- Disaster management
- Climate studies
- Infrastructure monitoring

9. Penetration Capability

Radar wavelength affects penetration.

NISAR uses:

- L-band → penetrates vegetation canopy and dry soil
- S-band → more sensitive to surface features

This dual-frequency capability is a unique characteristic of NISAR data.

10. All-Weather and Day-Night Imaging

Unlike optical images, backscatter images:

- Are independent of sunlight
- Are unaffected by clouds or monsoon

This is crucial in regions like India where cloud cover is frequent.

Summary of Key Characteristics

1. Brightness indicates radar reflectivity
2. Strong dependence on roughness and moisture
3. Multiple scattering mechanisms
4. Presence of speckle noise
5. Multi-polarization capability
6. Quantitative radiometric values (σ^0)
7. Time-series change detection
8. Penetration through vegetation and clouds

Add Multidimensional Data – Individual Variables

You can also add variables from the GSLC files as multidimensional rasters. Again, you will only be able to access the amplitude measurements in the SLC variables, but there are other variables that are better suited to this approach.

The other gridded frequency variable is the validity mask, but the variables in the /metadata/radarGrid path can also be visualized in ArcGIS Pro. These variables include along-track unit vectors, elevation angles, ground track velocity, incidence angles, LOS unit vectors, slant range, and Zero Doppler Azimuth time.

To add individual variables, click the drop-down arrow on the right side of the Add Data button, and select Add Multidimensional Data. Navigate to the GSLC HDF5 file and select it to view the available variables.

Click one or more of the variables listed, and leave the Output Configuration set to the default of Multidimensional Raster. Click the OK button to add the variable(s) to the map. If you select multiple variables, they will each be added as individual layers.



The screenshot displays the ENVI 5.1 software interface. The top menu bar includes Project, Map, Insert, Analysis, View, Edit, Imagery, Share, Planet Imagery, Help, Range, Raster Layer, and Data. The Imagery menu is highlighted with a red box. Below the menu, the Imagery toolbar contains icons for New Workspace, Georeference, Raster Functions (highlighted with a red box), Function Editor, Change Detection, Classification Wizard, Classification Tools, Deep Learning Tools, Mensuration Tools, Results, Process, Indices, Pixel Editor, Image Information, Create Hosted Imagery, and Video Search. The main workspace shows a topographic map with a large, dark, rectangular SAR image overlay. The SAR image is labeled 'Complex' in the Raster Functions panel. The Raster Functions panel on the right lists various functions, with 'Complex' selected. The bottom status bar shows the current map coordinates and scale.

This opens the dialog window for the Complex raster function. Select the same layer (focused SLC variable) for both the real and imaginary inputs. The layer must have been added as a Multiband Raster.

For the value type, select either Amplitude or Phase, then click the button to Create new layer. This will add a virtual layer to your map project. Alternatively, select Save As from the drop-down menu instead of Create new layer if you want to save the amplitude or phase output a new file for use in other projects.

Factors Affecting Strength of Backscattered Energy Signal:

1. Surface Roughness (Most Important Factor)

Surface roughness is measured relative to the **radar wavelength**.

- **Smooth surfaces:**
Reflect energy away from the radar → weak return → dark pixels
Example: calm water, smooth sand
- **Rough surfaces:**
Scatter energy in many directions → strong return → bright pixels
Example: rocky terrain, ploughed fields

Keypoint:

A surface may appear smooth at L-band but rough at shorter wavelengths.

2. Dielectric Constant (Moisture Content)

The dielectric constant controls how strongly a material reflects radar energy.

- Dry soil → low dielectric constant → low backscatter
- Wet soil → high dielectric constant → strong backscatter

Water is a very strong reflector, which is why:

- Flooded fields appear bright or dark depending on surface condition
- Soil moisture mapping is possible using radar

3. Angle of Incidence (Look Angle)

The **incidence angle** is the angle between the radar beam and the ground surface.

Effects:

- Small incidence angle → stronger backscatter
- Large incidence angle → weaker backscatter

Terrain slope also modifies this:

- Slopes facing radar → bright
- Slopes facing away → dark

This is called **topographic effect**.

4. Wavelength of Radar Signal

Radar wavelength determines penetration and interaction depth.

- Long wavelength (L-band, used in NISAR)
 - Penetrates vegetation and dry soil
 - Sensitive to trunk and ground structure
- Short wavelength
 - Sensitive to surface features only

Thus wavelength affects:

- Vegetation response
- Soil moisture detection
- Surface roughness sensitivity

5. Polarization of Radar Waves

Radar signals can be transmitted and received in different polarizations:

- HH
- VV
- HV
- VH

Different polarizations interact differently with:

- Vegetation
- Soil
- Buildings

Example:

- Cross polarization → vegetation structure
- Co polarization → surface roughness

6. Surface Geometry and Structure

Shape and orientation of objects strongly affect backscatter.

Three important mechanisms:

Surface Scattering

Flat or gently rough surfaces.

Volume Scattering

Occurs in vegetation and forest canopy.

Double-Bounce Scattering

Occurs in:

- Urban buildings
- Tree trunks over wet ground

Double-bounce produces very strong backscatter.

7. Vegetation Cover

Vegetation influences radar signals by:

- Absorbing energy
- Scattering energy multiple times
- Changing polarization

Dense forests usually give:

- Moderate to high backscatter
- Strong cross-polarized return

8. Surface Orientation Relative to Radar

Backscatter depends on whether the surface:

- Faces the radar
- Is tilted away

Effects:

- Foreshortening
- Layover
- Shadow (very low backscatter)

These are common in mountainous regions.

9. Frequency of Observation and Temporal Conditions

Backscatter can change with:

- Rainfall
- Crop growth stage
- Soil moisture variation

- Snow or frost

Therefore radar images vary over time even for the same area.

10. Atmospheric and System Factors (Secondary Effects)

Radar signals are less affected by atmosphere compared to optical sensors, but small effects may occur due to:

- Heavy rainfall attenuation
- Instrument calibration
- Noise level of sensor

These are usually corrected during processing.

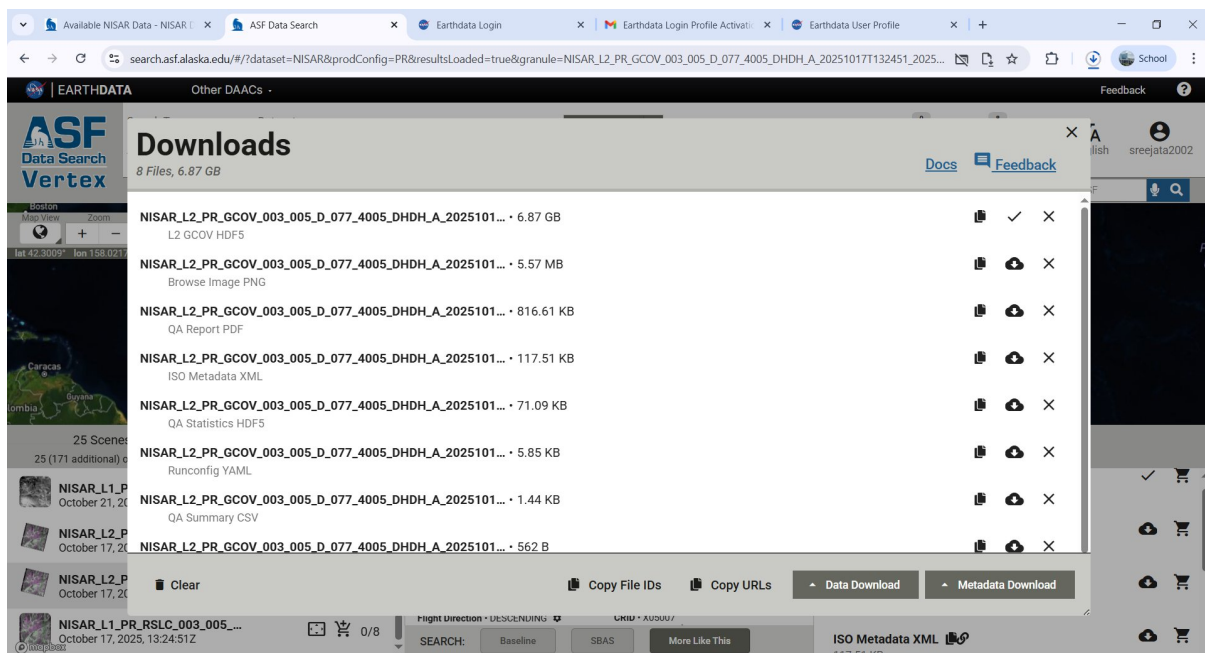
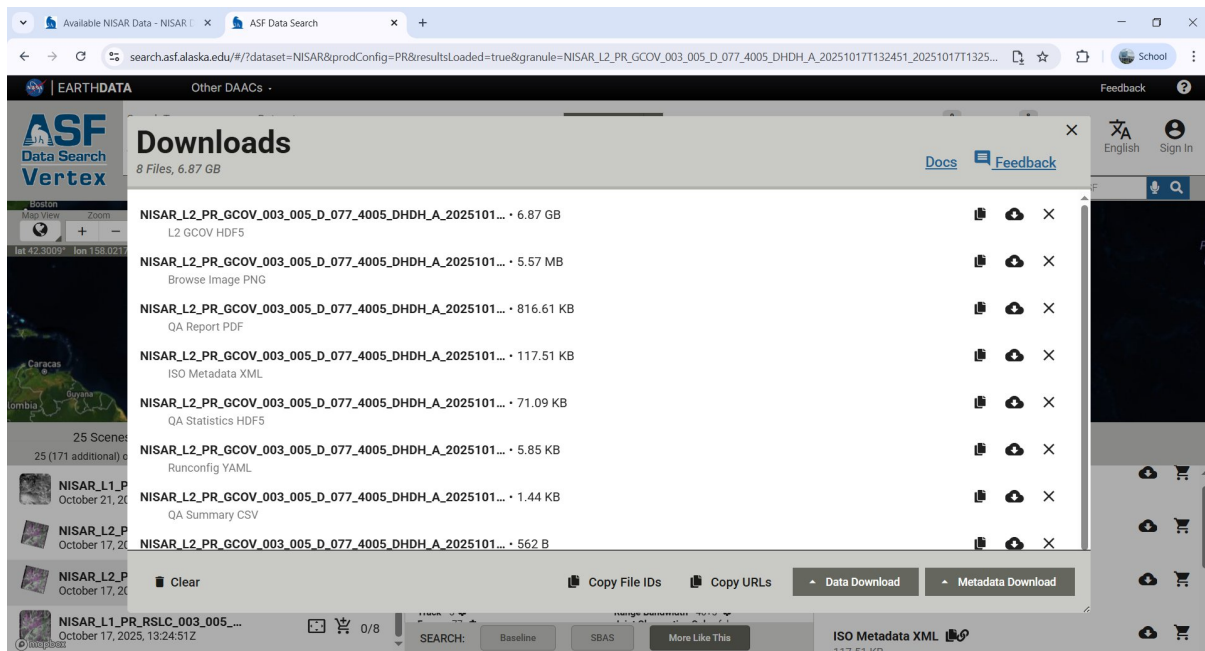
Summary Table

Factor	Effect on Backscatter
Surface roughness	Rough → high return
Soil moisture	Wet → high return
Incidence angle	Small angle → strong return
Wavelength	Longer wavelength → deeper penetration
Polarization	Different interaction with vegetation and soil
Vegetation	Volume scattering increases return
Geometry & slope	Facing radar → bright
Surface structure	Buildings give strong double-bounce
Time variation	Rain and crop growth change signal

GCOV L2 SAR image is a **Level-2 Ground Range Covariance Matrix product** generated from SAR data of missions such as **NISAR**, developed by **NASA** and **Indian Space Research Organisation (ISRO)**.

This product contains **polarimetric backscatter information** in the form of a covariance matrix, which preserves relationships between different polarizations and allows detailed interpretation of surface properties.

Below are the **results and observations typically obtained from a GCOV L2 SAR image**.



1. Nature of GCOV L2 SAR Product

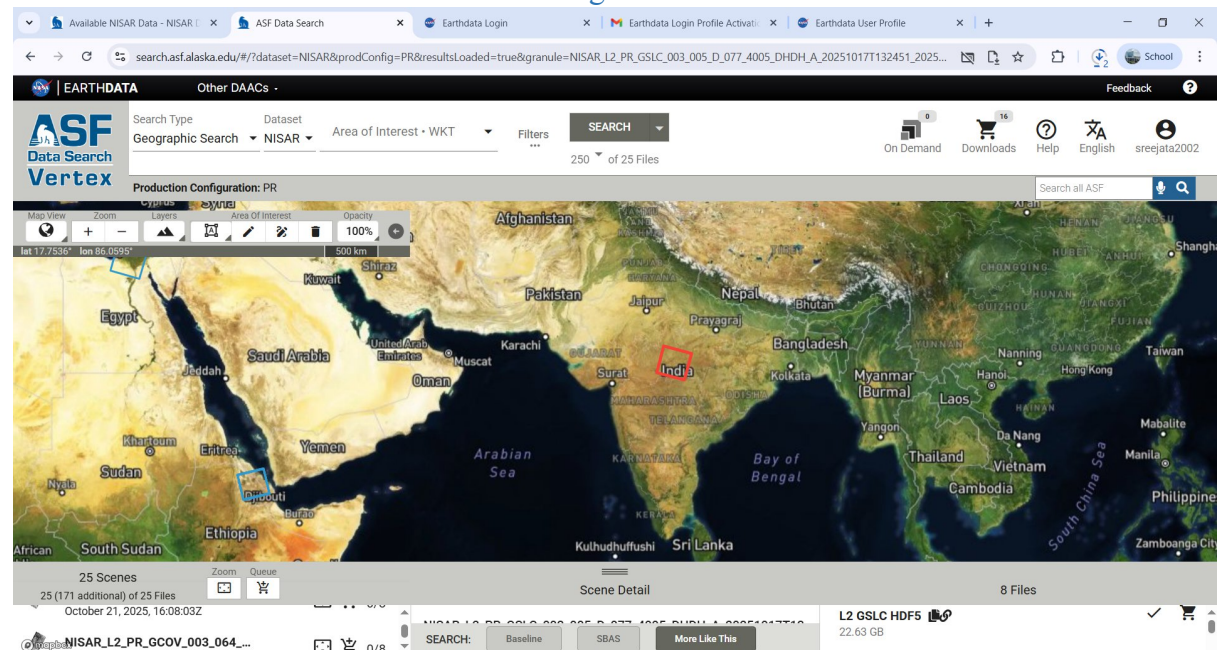
GCOV Level-2 product provides:

- Radiometrically calibrated backscatter
- Ground-range geometry (terrain corrected)
- Multi-polarization covariance matrix elements
- Reduced speckle compared to SLC products

This makes it suitable for:

- Land-cover mapping
- Surface characterization
- Polarimetric analysis

3. Results Observed in GCOV L2 SAR Images



A. Clear Differentiation of Land-Cover Types

One of the primary observations is that different land-cover classes show distinct backscatter responses.

Typical observations:

Water bodies

- Very low backscatter
- Appear dark
- Smooth surface reflects radar away

Urban areas

- Very high backscatter
- Bright appearance
- Due to double-bounce scattering

Agricultural fields

- Moderate backscatter

- Varies with crop stage and moisture

Forest areas

- Moderate to high backscatter
- Volume scattering dominates

Result:

GCOV images allow accurate **land-cover classification**.

3. Polarimetric Observations

Since GCOV contains covariance matrix elements, it preserves:

- Co-polarized signals (HH, VV)
- Cross-polarized signals (HV, VH)

Observations:

- Cross-pol channels show strong response in vegetation.
- Co-pol channels show stronger response from bare soil and urban areas.

Result:

Better discrimination between:

- Cropland vs forest
- Built-up vs barren land

4. Reduced Speckle and Improved Interpretability

Compared to SLC data:

Observations:

- Speckle noise is reduced due to multilooking.
- Texture becomes smoother.
- Features such as rivers, roads, and field boundaries become clearer.

Result:

GCOV Level-2 images are easier to interpret visually and for classification.

5. Soil Moisture and Surface Roughness Patterns

Observations in agricultural and barren regions:

- Wet soil → brighter pixels
- Dry soil → darker pixels
- Ploughed fields show higher backscatter than smooth fields

Result:

GCOV images help in:

- Soil moisture mapping
- Agricultural monitoring

6. Topographic Effects

In hilly or mountainous terrain:

Observations:

- Slopes facing radar appear bright.
- Slopes facing away appear dark.
- Radar shadow regions appear very dark.

Result:

Terrain orientation significantly influences pixel brightness.

7. Texture and Structural Information

GCOV data preserves texture information useful for:

Observations:

- Forest areas show coarse texture.
- Urban areas show strong bright spots and edges.
- Agricultural areas show uniform patterns.

Result:

Texture analysis improves:

- Land-use classification
- Object detection

8. Temporal Change Detection Capability

When comparing two GCOV images from different dates:

Observations:

- Flooded regions show sudden decrease in backscatter.
- Harvested fields show reduction in vegetation signal.
- Urban expansion appears as new bright regions.

Result:

Useful for:

- Disaster monitoring
- Crop monitoring

- Urban growth studies

9. Quantitative Observations (Backscatter Values)

Backscatter in GCOV images is calibrated and expressed in σ^0 (dB).

Typical observations:

Surface Type Backscatter Range

Water	−25 to −15 dB
Bare soil	−15 to −7 dB
Vegetation	−10 to −3 dB
Urban areas	−5 to +5 dB

Result:

Allows quantitative comparison between regions.

10. Polarimetric Decomposition Observations (Advanced Analysis)

Using GCOV covariance matrices, decomposition methods (Freeman–Durden, Cloude–Pottier) show:

Observations:

- Surface scattering dominant in bare land
- Volume scattering dominant in forests
- Double-bounce dominant in urban areas

Result:

Physical interpretation of scattering mechanisms becomes possible.

11. Accuracy and Reliability Observations

GCOV Level-2 images show:

- Geometrically corrected pixels
- Accurate geolocation
- Uniform radiometric calibration

Result:

Data can be directly used in GIS software like:

- QGIS

- SNAP
- ArcGIS

Summary of Major Results and Observations

From GCOV L2 SAR images we observe:

1. Clear separation of water, vegetation, soil, and urban areas
2. Reduced speckle compared to Level-1 data
3. Strong polarimetric information preserved
4. Sensitivity to soil moisture and roughness
5. Terrain effects visible in mountainous areas
6. Useful texture patterns for classification
7. Reliable quantitative backscatter values
8. Capability for change detection over time

Using the Command Line

```
file="path/to/input.h5"
subdataset1="${file}://science/LSAR/GCOV/grids/frequencyA/HHHH"
subdataset2="${file}://science/LSAR/GCOV/grids/frequencyA/numberOfLooks"
subdataset3="${file}://science/LSAR/GCOV/grids/frequencyA/rtcAreaNormalizat
ionFactorGamma0ToSigma0"
dimensions_args="-ul_lr upper_left_corner_x upper_left_corner_y
lower_right_corner_x lower_right_corner_y -ps pixel_size_x pixel_size_y"
gdal_merge.py $dimensions_args -seperate -of GTiff -o $(echo $file | sed
's/\.h5/\_multiband.tif/g')
```

Using Python

Here is the code to extract raster subsets in h5 and convert them into geotiff files.

```
import json
import subprocess
import re
from osgeo import gdal

file="NISAR_L2_PR_GCOV_001_005_A_219_4020_SHNA_A_20081012T060910_20081012T0
60926_P01101_F_N_J_001.h5"
actual_json = json.loads(subprocess.check_output(f"gdalinfo -json {file}",
shell=True))
datasets = actual_json['metadata']['SUBDATASETS']
names = [key for key in datasets.keys() if re.match('SUBDATASET_[0-
9]*_NAME', key)]
for name in names:
    subdata = f"{datasets[name].replace('HDF5','NETCDF')}"
    if gdal.Open(subdata).GetProjectionRef() != '':
        outfile = f"{subdata.replace('/', '_')}.tif"
        subprocess.run(f'gdal_translate -of GTiff {subdata} {outfile}',
shell=True)
```

We also provide the code to combine all raster subdatasets with the same datatype, extent, size, and projection in the h5 into multiple-band geotiff files. We notice that

the combined multiple band geotiff files do not include the metadata for each band in the geotiff files.

```
import json
import subprocess
import re
import numpy as np
from osgeo import gdal

def combine_geotiff(filelist, outfile):
    '''
    Merge the geotiff files with the same data type, extent, and projection
    into a multiple band geotif file
    :param filelist: list of geotiff files
    :param outfile: filename of the output file
    :return: None
    '''
    bandmetalist = [gdal.Info(file, format='json')['bands'] for file in
filelist]
    bandnumlist = [len(item) for item in bandmetalist]
    bandnum = int(np.array(bandnumlist).sum())
    filenum = len(filelist)
    for i in range(filenum):
        ds = gdal.Open(filelist[i])
        if i == 0:
            # create output raster file
            driver = gdal.GetDriverByName("GTiff")
            outdata = driver.Create(outfile, ds.RasterXSize,
ds.RasterYSize, bandnum, ds.GetRasterBand(1).DataType )
            outdata.SetGeoTransform(ds.GetGeoTransform()) ##sets same
geotransform as input
            outdata.SetProjection(ds.GetProjection()) ##sets same
projection as input
            bandnum_of_file = ds.RasterCount
            fmeta = ds.GetMetadata()
            for j in range(1, bandnum_of_file + 1):
                band = ds.GetRasterBand(j)
                outdata.GetRasterBand(i*bandnum_of_file +
j).WriteArray(band.ReadAsArray())
                fmeta.update(band.GetMetadata())
                outdata.GetRasterBand(i*bandnum_of_file + j).SetMetadata(fmeta)
            ds = None
            outdata = None
file="NISAR_L2_PR_GCOV_001_005_A_219_4020_SHNA_A_20081012T060910_20081012T0
60926_P01101_F_N_J_001.h5"
actual_json = json.loads(subprocess.check_output(f"gdalinfo -json {file}",
shell=True))
datasets = actual_json['metadata']['SUBDATASETS']
names = [key for key in datasets.keys() if re.match('SUBDATASET_[0-
9]*_NAME', key)]
datalist=[]
for name in names:
    subdata = f"{datasets[name].replace('HDF5','NETCDF')}"
    ds = gdal.Open(subdata)
    if ds.GetProjectionRef() != '':
        size = f'{ds.RasterXSize}x{ds.RasterYSize}'
        outfile = f"{subdata.replace('/', '_')}.tif"
        outfile = outfile.split(":")[2].replace('_', '')
        mo=outfile.replace(".tif", "")
```

```

        subprocess.run(f"gdal_translate -of GTiff -ot Float32 -mo Band={mo}
{subdata} {outfile}", shell=True)
        datalist.append({"filename":outfile, "size":size})
sizelist = [item['size'] for item in datalist]
uniqvals= np.unique(sizelist)
for item in uniqvals:
    itemlist = [ dataitem['filename'] for dataitem in datalist if
dataitem['size'] == item ]
    mb_outfile = subdata.split(":")[1].replace('.h5',f'_{item}.tif')
    mb_outfile = mb_outfile.replace(' ','')
    combine_geotiff(itemlist, mb_outfile)

```

Python code for GSLC Product in Jupyter Lab:

Python code for NISAR GSLC Product:

```

# =====
# NISAR GSLC Backscatter Processing and Export to GeoTIFF
#
# This script reads a NISAR GSLC HDF5 file, extracts radar data,
# computes the radar backscatter intensity, converts it into
# decibel (dB) scale, visualizes the result, and exports the
# processed image as a GeoTIFF file that can be opened in GIS
# software such as ArcGIS or QGIS.
#
# GSLC stands for Geocoded Single Look Complex product.
# It contains complex SAR data (real and imaginary components)
# that represent radar amplitude and phase information.
#
# The workflow performed in this code:
# 1. Load required Python libraries
# 2. Open the NISAR GSLC HDF5 file
# 3. Explore the dataset structure
# 4. Extract complex SAR data (VV polarization)
# 5. Compute radar intensity from complex values
# 6. Convert intensity to backscatter in decibel scale
# 7. Visualize the radar image
# 8. Compute spatial resolution from coordinate arrays
# 9. Create georeferencing transform
# 10. Export the processed image as a GeoTIFF raster
# =====
#
# -----
# Import libraries required for radar data processing
# -----
#
# h5py library is used to read HDF5 files such as NISAR products
import h5py
#
# numpy is used for numerical calculations and array operations
import numpy as np
#
# rasterio is used for writing geospatial raster files (GeoTIFF)
import rasterio

```

```

# function used to define spatial reference transform of raster
from rasterio.transform import from_origin

# matplotlib is used to visualize the SAR image in Jupyter
import matplotlib.pyplot as plt

# -----
# Step 1: Define the path of the NISAR GSLC data file
# Replace the filename below with your GSLC file location
# -----

file_path = "NISAR_GSLC_file.h5"

# -----
# Step 2: Open the GSLC file in read mode
# -----

# 'with' statement ensures the file is automatically closed
# after reading, preventing memory issues
with h5py.File(file_path, 'r') as f:

    # Print message to indicate dataset listing
    print("Datasets inside the NISAR GSLC file:")

    # Function to print dataset names
    # It will be used to explore the internal structure
    def printname(name):
        print(name)

    # Visit every group and dataset inside the HDF5 file
    # This helps identify where radar data is stored
    f.visit(printname)

    # -----
    # Step 3: Extract complex SAR data for VV polarization
    #
    # GSLC data contains complex values representing
    # radar amplitude and phase.
    #
    # Complex number format:
    # S = Real + i(Imaginary)
    #
    # Backscatter intensity is derived from the magnitude
    # of the complex number.
    # -----

    complex_data = f['/science/LSAR/GSLC/grids/frequencyA/VV'][:]

    # -----
    # Step 4: Extract coordinate arrays
    #
    # These arrays store the spatial grid coordinates
    # of the radar image pixels.
    #
    # They are required to correctly georeference
    # the exported raster image.

```

```

# -----

x = f['/science/LSAR/GSLC/grids/frequencyA/xCoordinates'][:,]
y = f['/science/LSAR/GSLC/grids/frequencyA/yCoordinates'][:,]

# -----
# Step 5: Compute radar intensity
#
# GSLC data is complex-valued (real + imaginary).
# Radar intensity is calculated using the magnitude squared.
#
# Intensity =  $|S|^2$ 
# where S is the complex radar signal.
# -----

intensity = np.abs(complex_data) ** 2

# -----
# Step 6: Convert radar intensity to backscatter in decibels
#
# Radar backscatter is usually expressed in logarithmic scale:
#
# Backscatter (dB) =  $10 * \log_{10}(\text{Intensity})$ 
#
# This conversion compresses the dynamic range and
# makes the radar image easier to interpret.
# -----

backscatter_db = 10 * np.log10(intensity)

# -----
# Step 7: Visualize the backscatter image
#
# The radar image is displayed as a grayscale image
# where brighter pixels represent stronger reflections.
# -----

plt.figure(figsize=(8,6))                                # Create figure
window                                                    # window
plt.imshow(backscatter_db, cmap='gray')                  # Display radar image
plt.colorbar(label='Backscatter (dB)')                   # Add scale bar
plt.title('NISAR GSLC Backscatter (VV Polarization)')    #
plt.show()                                                # Show the image

# -----
# Step 8: Calculate pixel resolution
#
# Pixel size is determined by the difference between
# consecutive coordinate values.
# -----

pixel_size_x = abs(x[1] - x[0])                          # Horizontal
resolution                                         # resolution
pixel_size_y = abs(y[1] - y[0])                      # Vertical resolution

```

```

# -----
# Step 9: Create spatial transformation
#
# The transform defines how pixel coordinates
# correspond to real-world geographic coordinates.
# -----

transform = from_origin(x.min(), y.max(),
                        pixel_size_x, pixel_size_y)

# -----
# Step 10: Define output GeoTIFF file name
# -----

output_file = "nisar_gslc_backscatter_vv.tif"

# -----
# Step 11: Export the processed radar image as GeoTIFF
#
# GeoTIFF stores both raster data and spatial metadata.
# This allows the file to be directly opened in GIS software.
# -----

with rasterio.open(
    output_file,                                # Output raster file
    'w',                                         # Write mode
    driver='GTiff',                             # GeoTIFF format
    height=backscatter_db.shape[0],             # Number of rows
    width=backscatter_db.shape[1],             # Number of columns
    count=1,                                    # Single raster band
    dtype=backscatter_db.dtype,                # Data type
    crs='EPSG:4326',                           # Coordinate system (WGS84)
    transform=transform                         # Spatial transformation
) as dst:

    # Write the backscatter array to raster band 1
    dst.write(backscatter_db, 1)

# -----
# Step 12: Display completion message
# -----

print("GeoTIFF Exported Successfully: nisar_gslc_backscatter_vv.tif")
print("The file can now be opened in GIS software.")

```

What This Code Does (Conceptual Workflow)

This program performs the following **SAR data processing steps**:

1. Opens the **NISAR GSLC radar dataset stored in HDF5 format**.
2. Extracts **complex SAR signal values for VV polarization**.
3. Computes **radar intensity from complex data**.
4. Converts the intensity into **backscatter in decibel scale**.
5. Displays the radar image for verification.
6. Calculates pixel resolution using coordinate arrays.
7. Creates spatial referencing information.
8. Exports the processed raster as a **GeoTIFF file**.

Python code for GCOV Data product

```
<> Python
```

```
# =====  
# NISAR GCOV Backscatter Extraction and Export to GeoTIFF  
# This code reads NISAR GCOV HDF5 data, converts sigma0 to  
# backscatter in dB, visualizes it, and exports it to GeoTIFF  
# so it can be opened in ArcGIS or other GIS software.  
# =====  
  
# Import library for reading HDF5 files (NISAR data format)  
import h5py  
  
# Import numerical computation library for array operations  
import numpy as np  
  
# Import rasterio for creating and writing GeoTIFF raster files  
import rasterio  
  
# Import function to define spatial transform of raster  
from rasterio.transform import from_origin  
# Import matplotlib to visualize the SAR image  
import matplotlib.pyplot as plt  
  
# -----  
# STEP 1: Specify the path of the NISAR GCOV HDF5 file  
# Replace this filename with the location of your NISAR file  
# -----  
  
file_path = "NISAR_GCOV_file.h5"  
  
# -----  
# STEP 2: Open the NISAR HDF5 file and read datasets  
# -----  
  
with h5py.File(file_path, 'r') as f:  
  
    # Print all dataset paths in the file to understand structure  
    print("Datasets inside the NISAR file:")
```

```

# Define a function to print dataset names
def printname(name):
    print(name)

# Visit all groups and datasets inside the HDF5 file
f.visit(printname)

# -----
# Extract sigma0 backscatter data for VV polarization
# Path may change slightly depending on NISAR product
# -----

sigma_vv = f['/science/LSAR/GCOV/grids/frequencyA/sigma0_vv'][:, :]

# -----
# Extract x and y coordinate arrays of the image grid
# These represent spatial positions of pixels
# -----

x = f['/science/LSAR/GCOV/grids/frequencyA/xCoordinates'][:, :]
y = f['/science/LSAR/GCOV/grids/frequencyA/yCoordinates'][:, :]

# -----
# STEP 3: Convert sigma0 to backscatter in decibels (dB)
# SAR backscatter is usually expressed in logarithmic scale
# Formula: Backscatter(dB) = 10 * log10(sigma0)
# -----

sigma_vv_db = 10 * np.log10(sigma_vv)

# -----
# STEP 4: Display the backscatter image using matplotlib
# This helps visualize radar intensity values
# -----

plt.figure(figsize=(8,6))                                # Define size of the plot
plt.imshow(sigma_vv_db, cmap='gray')                      # Display raster as grayscale image
plt.colorbar(label='Backscatter (dB)')                    # Add colorbar showing dB values
plt.title('NISAR Backscatter VV Polarization')            # Add title to the plot
plt.show()                                                 # Show the image

```

```

# -----
# STEP 5: Calculate pixel size (spatial resolution)
# Difference between adjacent coordinates gives resolution
# -----

pixel_size_x = abs(x[1] - x[0])          # Compute pixel width
pixel_size_y = abs(y[1] - y[0])          # Compute pixel height

# -----
# STEP 6: Create raster transform
# This defines how raster pixels map to geographic space
# -----

transform = from_origin(x.min(), y.max(), pixel_size_x, pixel_size_y)

# -----
# STEP 7: Define output GeoTIFF file name
# -----

output_file = "nisar_backscatter_vv.tif"

# GeoTIFF format can be directly opened in GIS software
# -----

with rasterio.open(
    output_file,          # Output file name
    'w',                  # Write mode
    driver='GTiff',        # File format = GeoTIFF
    height=sigma_vv_db.shape[0], # Number of rows in raster
    width=sigma_vv_db.shape[1], # Number of columns in raster
    count=1,               # Number of bands (single band)
    dtype=sigma_vv_db.dtype, # Data type of pixel values
    crs='EPSG:4326',        # Coordinate reference system (WGS84)
    transform=transform     # Spatial transformation matrix
) as dst:

    # Write backscatter array into raster band 1
    dst.write(sigma_vv_db, 1)

```

```

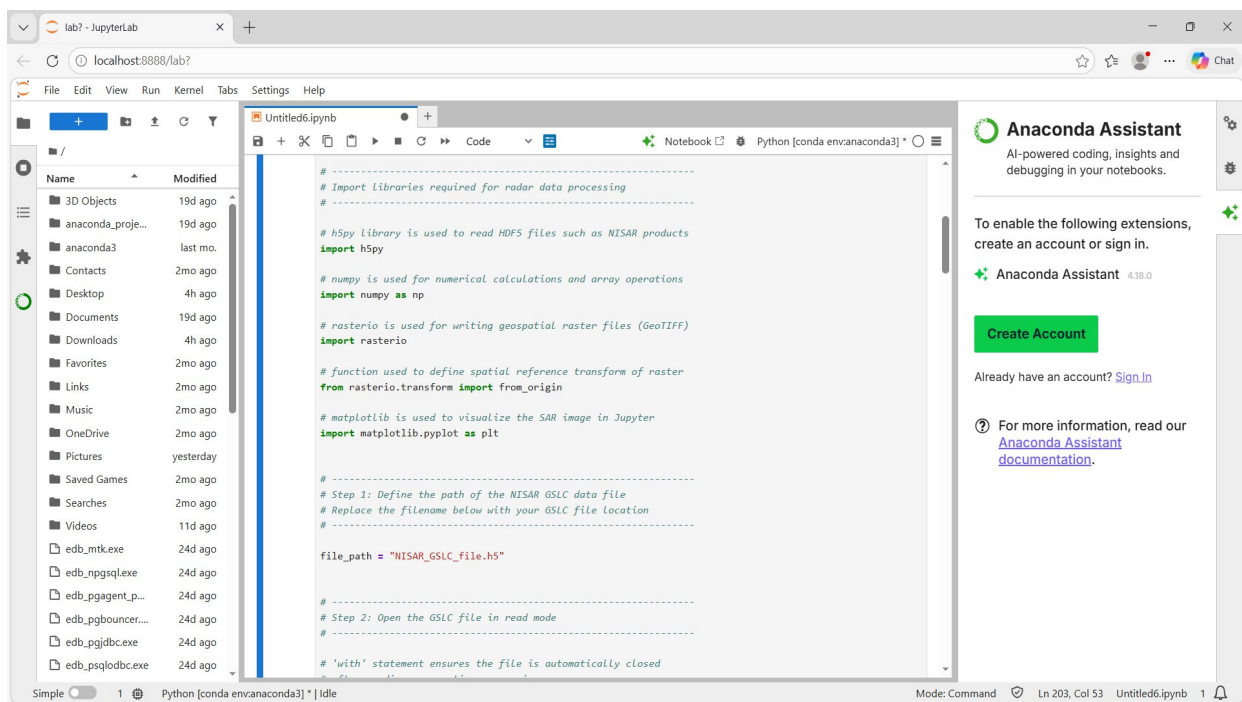
# -----
# STEP 9: Print confirmation message
# -----

print("GeoTIFF Exported Successfully: nisar_backscatter_vv.tif")

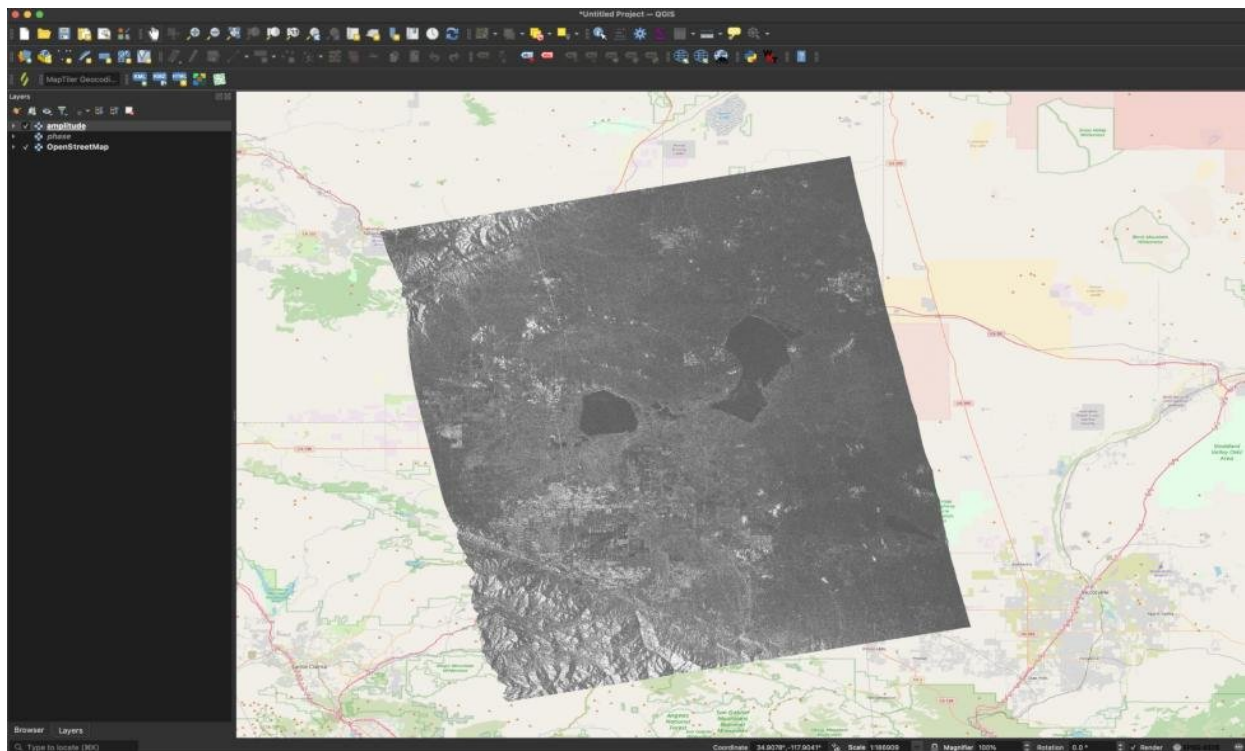
# -----
# STEP 10: The exported GeoTIFF can now be opened in GIS
# -----

print("You can now open the GeoTIFF in GIS software like ArcGIS.")

```

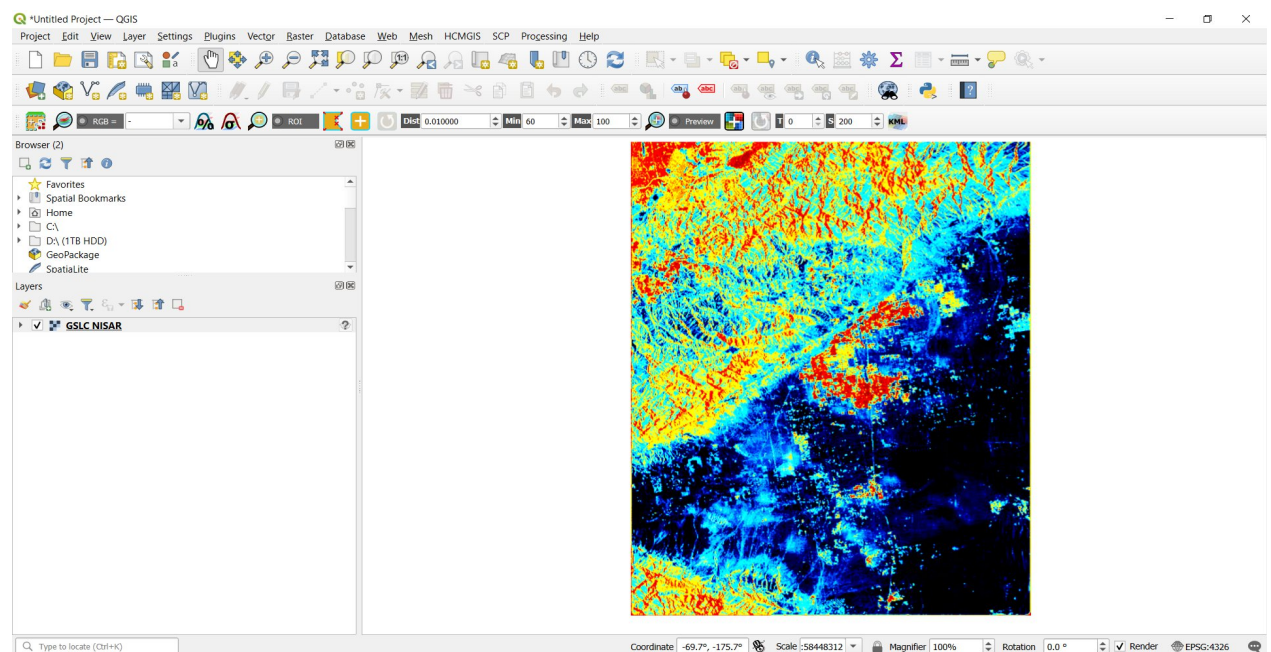
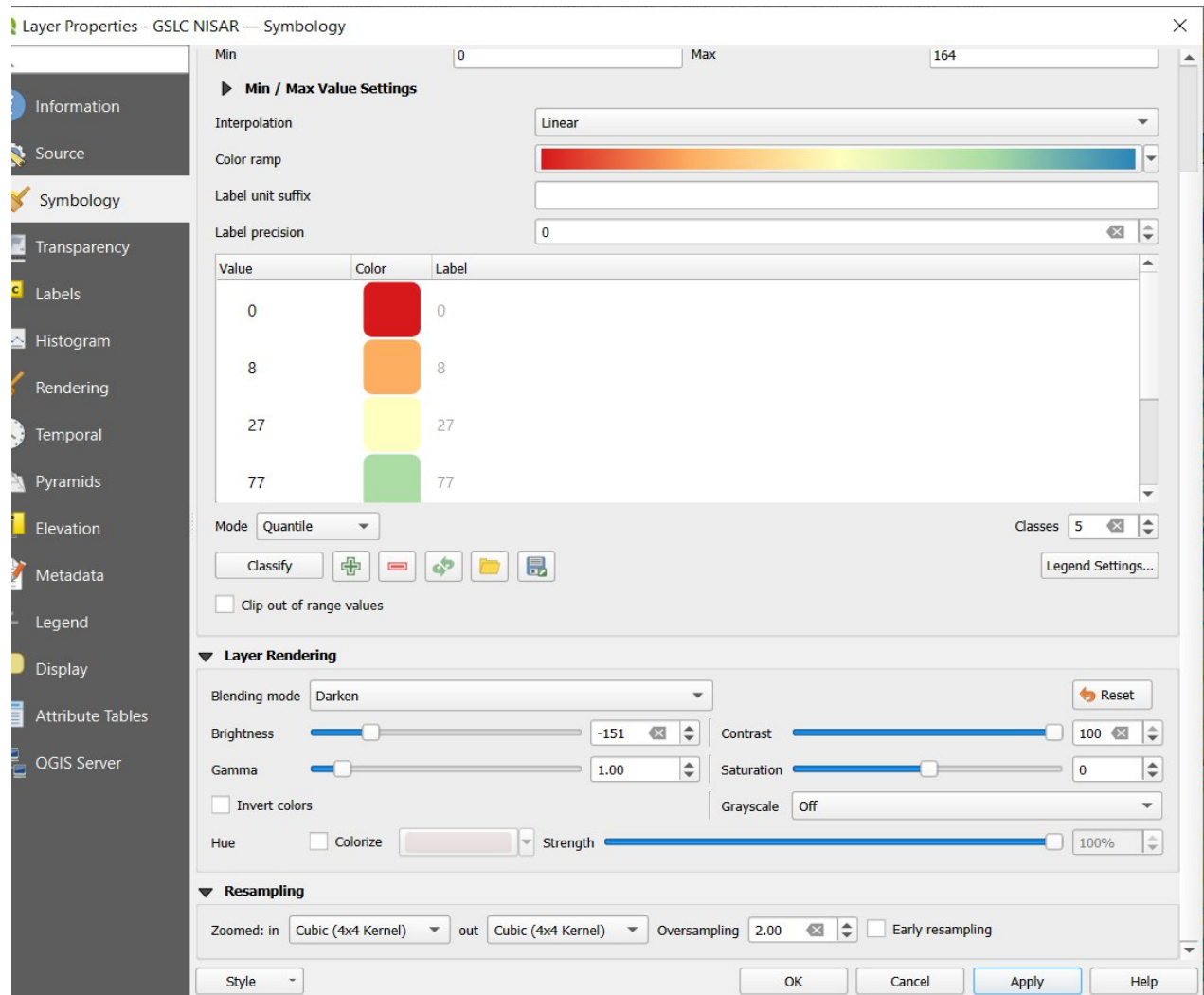


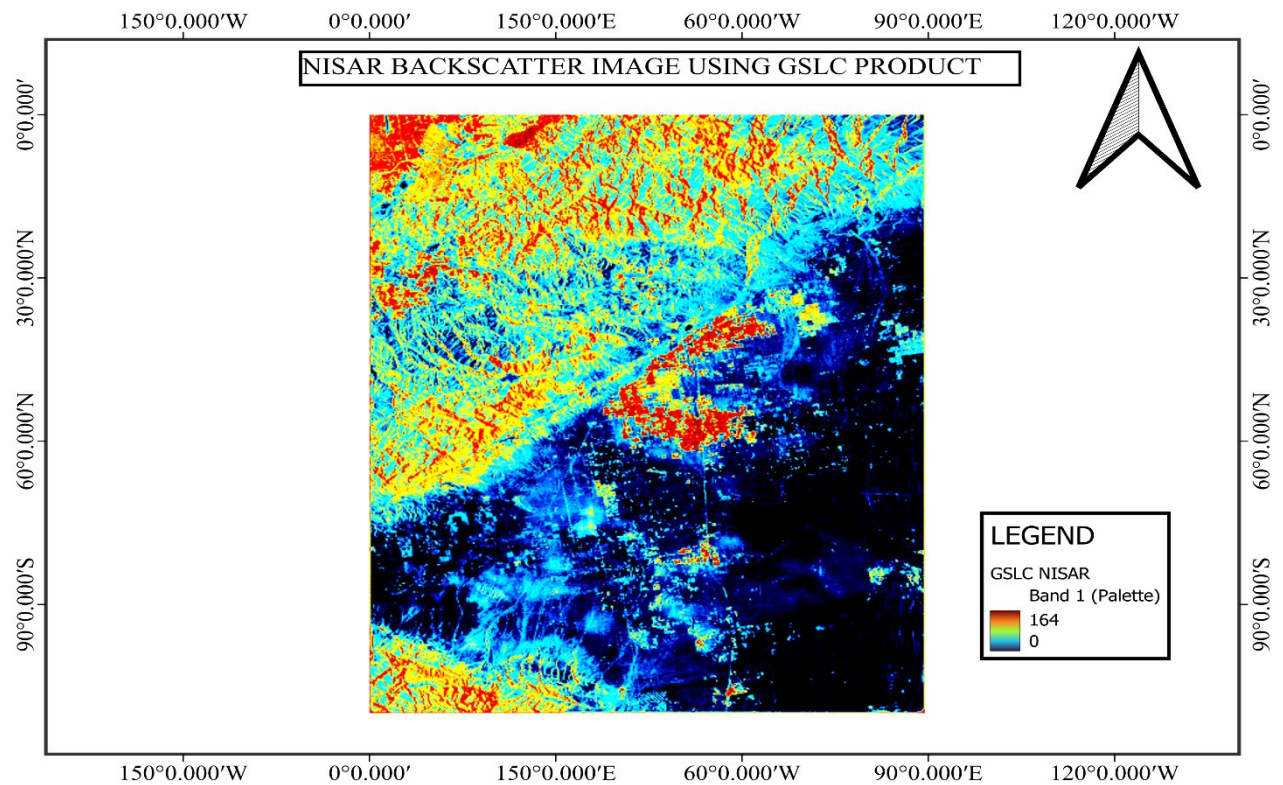
NISAR Backscatter code in Jupyter Lab to derive backscatter image



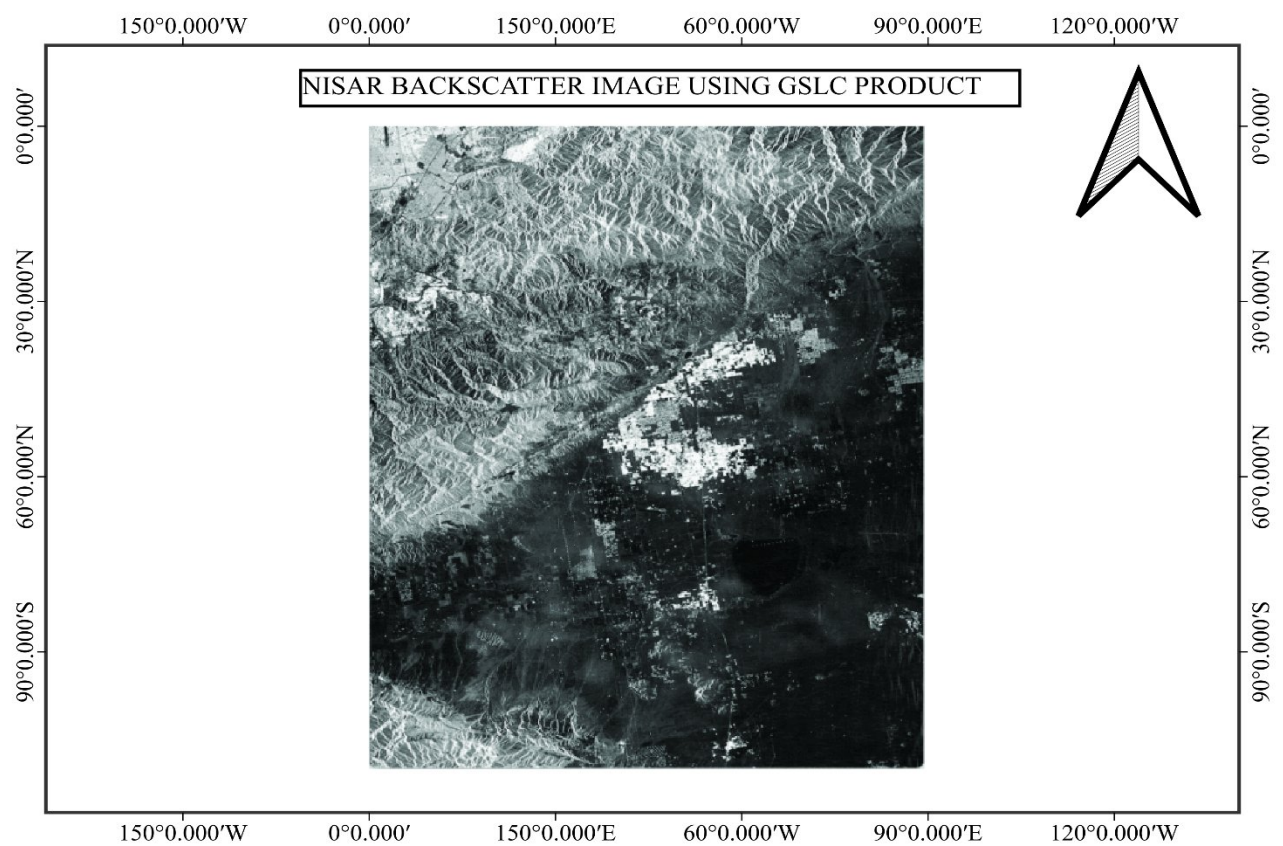
Convert the single-band Complex32 dataset into a two-band Float32 GeoTIFF file

```
from osgeo import gdal
import numpy as np
gdal.UseExceptions()
file="hh.tif"
outfile = "hh_polar.tif"
ds = gdal.Open(file)
band = ds.GetRasterBand(1)
data = band.ReadAsArray()
amp = abs(data)
phase = np.angle(data)
# create output raster file
driver = gdal.GetDriverByName("GTiff")
outdata = driver.Create(outfile, ds.RasterXSize, ds.RasterYSize, 2,
gdal.GDT_Float64)
outdata.SetGeoTransform(ds.GetGeoTransform())
outdata.SetProjection(ds.GetProjection())
outdata.SetMetadata(ds.GetMetadata())
outdata.GetRasterBand(1).WriteArray(amp)
outdata.GetRasterBand(1).SetMetadata({"bandname": "modulus"})
outdata.GetRasterBand(2).WriteArray(phase)
outdata.GetRasterBand(2).SetMetadata({"bandname": "argument"})
outdata = None
```





Map layout of NISAR Backscattered Image GSLC Data



Map layout of NISAR Backscattered Image GSLC Data

Conclusions:

The backscattered image obtained from **GCOV Level-2 SAR data of NISAR**, a mission jointly developed by **NASA** and **Indian Space Research Organisation (ISRO)**, provides detailed information about the Earth's surface based on radar reflectivity rather than visible light. In these images, brightness represents the strength of the returned radar signal, allowing clear identification of different land-cover types such as water bodies, vegetation, agricultural land, and urban areas.

One important observation from GCOV backscatter images is their sensitivity to surface characteristics such as roughness, moisture content, and structural geometry. Smooth surfaces like calm water appear dark due to weak backscatter, whereas rough surfaces, dense vegetation, and built-up regions appear brighter because they scatter radar energy more strongly. This property makes SAR data highly useful for land-surface characterization and environmental studies.

Another key conclusion is that GCOV data preserves polarimetric information through covariance matrix elements, enabling improved discrimination between different surface types. The availability of multiple polarizations helps in identifying scattering mechanisms such as surface scattering, volume scattering, and double-bounce scattering. As a result, GCOV images are particularly effective for vegetation analysis, soil moisture studies, and urban mapping.

GCOV Level-2 products also show reduced speckle noise and are radiometrically calibrated and geometrically corrected, which improves image interpretation and quantitative analysis. These features allow the data to be directly used in GIS platforms for mapping, classification, and change detection without extensive preprocessing, making it suitable for academic and research applications.

In summary, backscattered images derived from NISAR GCOV data provide reliable, all-weather, and day-night observations of the Earth's surface with high sensitivity to physical surface properties. Their ability to detect subtle changes over time makes them valuable for applications such as agriculture monitoring, disaster management, forest assessment, and land-use analysis, highlighting their importance in modern remote sensing and geospatial studies.

Conclusion: NISAR Level-2 GSLC Product:

The NISAR Geocoded Single-Look Complex (GSLC) is a highly advanced, Level-2 radar data product designed to provide precise, spatially accurate, and analysis-ready data for Earth observation. Derived from Level-1 Range-Doppler data (RSLC), the GSLC product bridges the gap between raw radar geometry and standard geographic mapping.

Key Characteristics of GSLC Data:

- **Preservation of Complex Data:** Unlike standard imagery that only shows visual intensity, GSLC data contains both **amplitude** (the strength of the radar backscatter) and **phase** (the precise timing of the radar wave's return).
- **Geocoded for GIS Readiness:** The data is geometrically corrected and projected onto a standard geographic grid (like UTM) using a Digital Elevation Model (DEM). This removes terrain distortions and allows the data to be easily integrated into standard GIS

software (like QGIS) without requiring the user to perform complex radar coordinate conversions.

- **Topographic Phase Flattening:** During processing, the phase shifts caused by the Earth's natural topography are calculated and removed. This "flattening" makes the data immediately ready for comparative analysis.

Primary Scientific Applications: The preservation of phase in a geocoded format makes the GSLC product foundational for **Interferometric SAR (InSAR)**. By cross-multiplying two GSLC products captured over the same area at different times, scientists can generate interferograms to measure millimeter-scale changes in the Earth's surface. This is critical for monitoring:

- **Solid Earth Dynamics:** Earthquakes, fault lines, and volcanic inflation.
- **Geohazards:** Land subsidence, groundwater depletion, and landslides.
- **Cryosphere Changes:** Glacial flow rates and ice sheet dynamics.

Relevance to the Visualizations: In the context of your recent map layouts, the visual maps you generated represent the **amplitude (intensity) component** of the GSLC data. By mapping these backscatter intensities (whether in grayscale or pseudocolor), you are able to clearly distinguish between rough terrain, urban infrastructure (high backscatter), and smooth surfaces like bare ground or water (low backscatter), all accurately mapped to their real-world global coordinates.